

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Homework 12: due on 4/25
  - Lab 5: due on 5/2
  - Project 4 and Quiz 4: due on 5/2
  - Bonus Opportunities
    - Bonus Homework: due on 5/2
    - Course Post-Test: due on 5/2
    - Bonus point for all when OMET response rate >= 80%
      - Currently at 25-28%
      - Deadline is Sunday 4/24

Previous Lecture …

- Disk arm scheduling
  - FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOk

Muddiest Points

- Just need to study the material more to be honest.
- DMA and CPU memory competition
- hardware view of interrupt
- Thanks for bringing in coffee and snacks for the final! It is much appreciated
- online vs offline problem
- What sort of questions about this material will be on the exam?
- In Sharing file, if the directory A is owner of file "a_file" and then this file is shared by directory B, Then we delete directory A, Are all files even shared files will be deleted and not more accessible, or we have one copy of "a_file " in directory of B?
- Can you please go over File Linking again?
- How do you read in the blocks from the disk file for project 4? I know to use fseek and fread but I don't know where to go from there.
- Can you explain the difference between CSCAN and CLOOK?

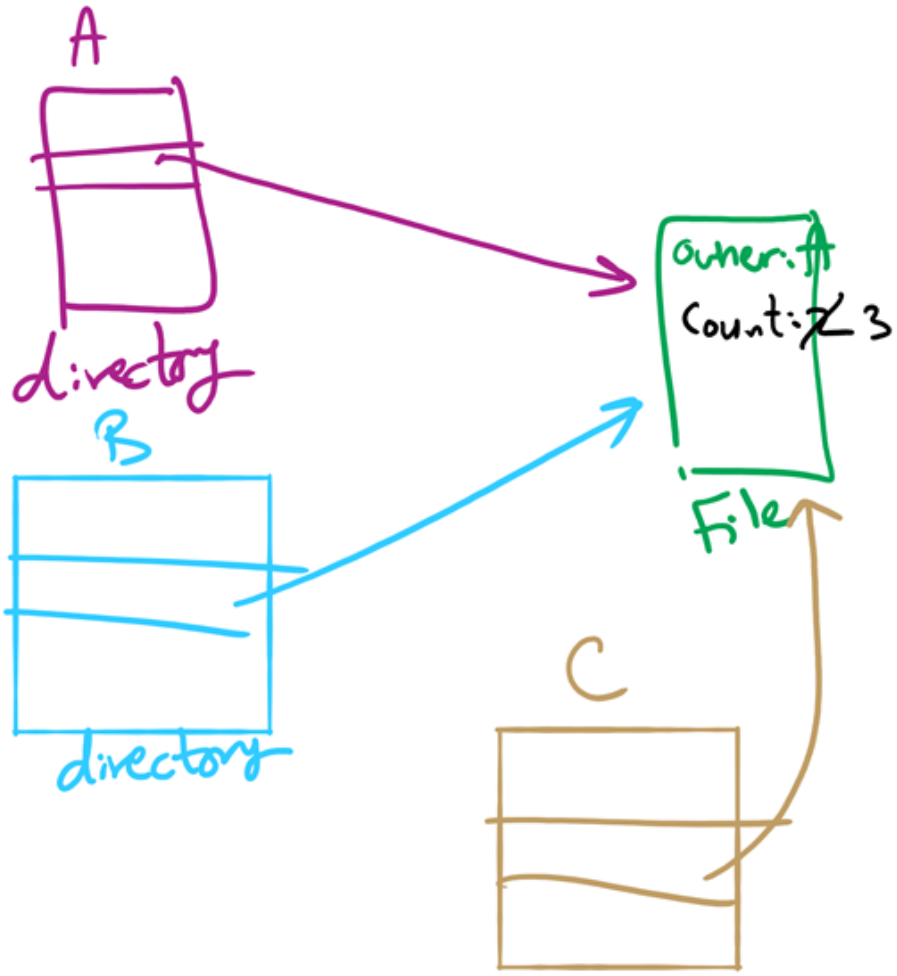
Journaling File System

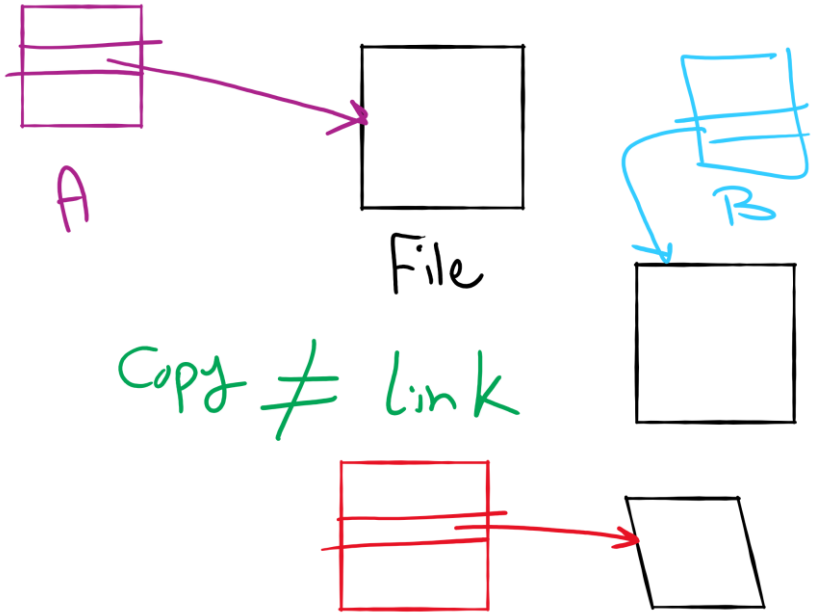
- In a regular file system, changes to files and directories result in multiple separate writes to disk
  - prone to power failures
- Write changes twice
  - first to an on-disk *journal*
    - for efficiency, journal can be put on SSD or NVRAM
    - data may or may not be written to the journal
  - second to main file system
- Examples
  - Windows NTFS
  - Linux ext3, ext4

Journaling File System

- Interaction of disk arm scheduling?
  - out-of-order writes
  - ext3 and ext4 force disk to flush its cache at certain points (barriers)
- Journaling vs. Log-structured file system
  - Journaling is not needed in LSFS

File Linking (left) vs. File Copying (right)





Max Partition Size

Block size	FAT-12	FAT-16	FAT-32
0.5 KB	2 MB		
1 KB	4 MB		
2 KB	8 MB	128 MB	
4 KB	16 MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

Max Partition Size

32-bit block no. (block no. size)

Max. Partition Size = $2^{\text{block no. size}} * \text{block size}$

FAT-12: $2^{12} * \frac{1}{2}\,kB = 2^{12} * 2^{9} = 2^{21} = 2\,MB$

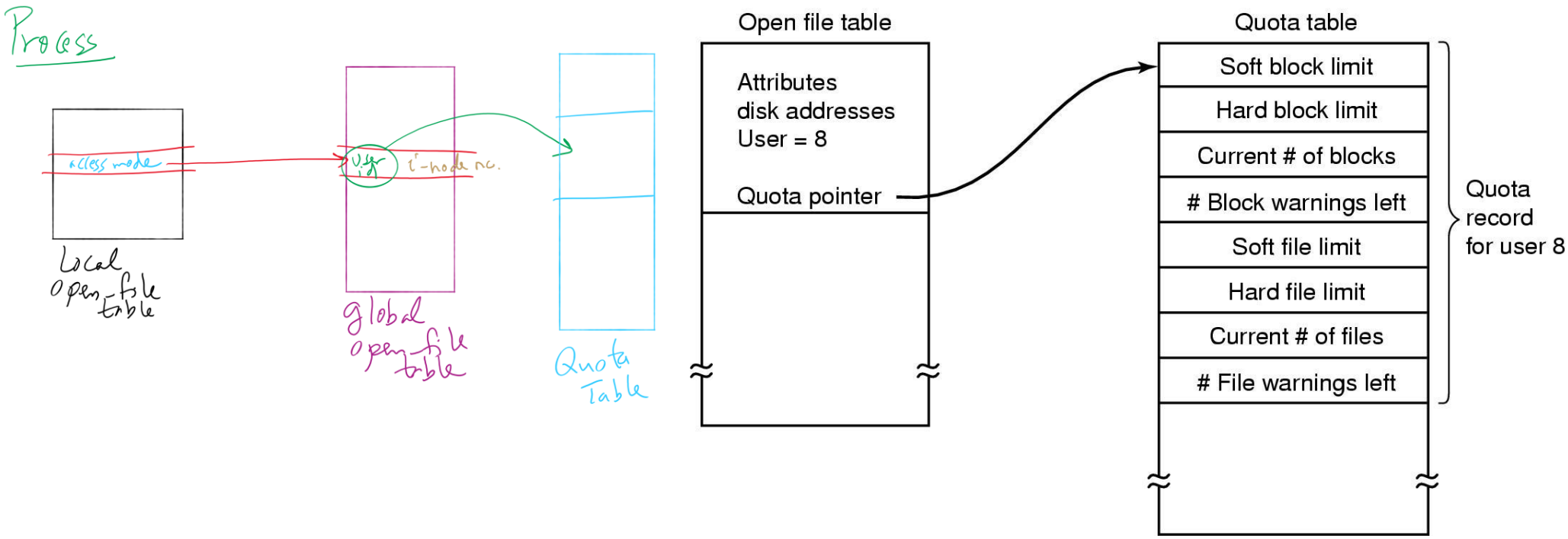
FAT-16: $2^{16} * 2\,kB = 2^{16} * 2^{11} = 2^{27} = 128\,MB$

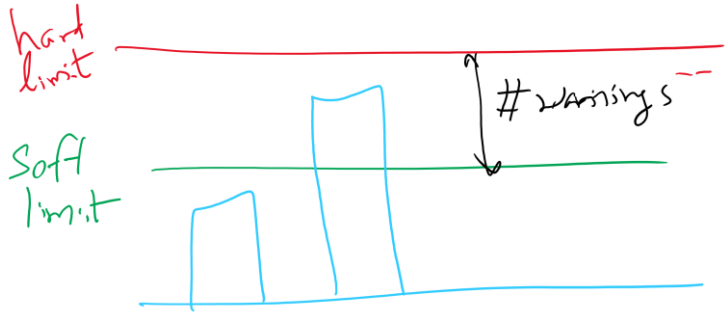
FAT-32: $2^{28} * 4\,kB = 2^{28} * 2^{12} = 2^{40} = 1\,TB$

sector no. size

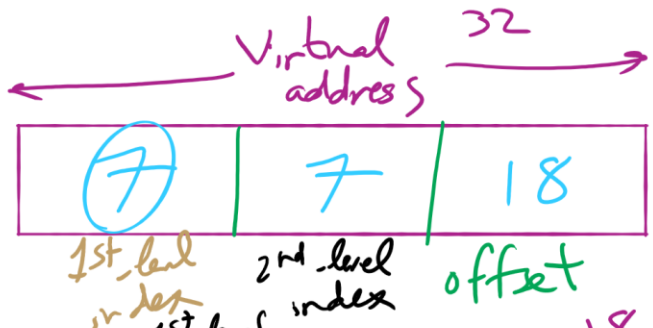
$2^{32} * 512 = 2^{32} * 512 = 2^{41} = 2\,TB$

File-related kernel structures: open file tables and disk quotas





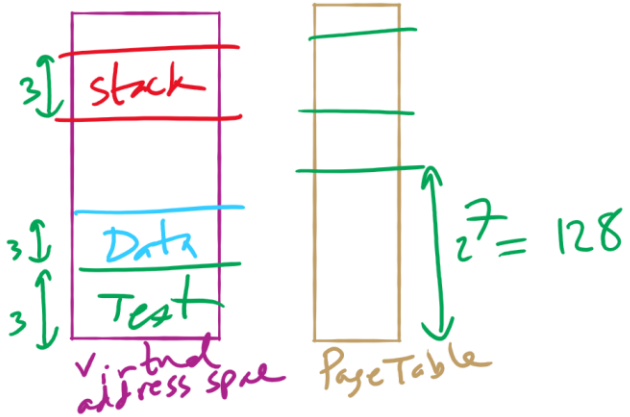
HW 10: Q 2-4



$\#PDEs = 2^{\text{1st_level index size}} = 2^7$

$2^8 KB = 2^{18}$ bytes

$\#PTEs$ in 2nd_level PageTable $= 2^{\text{2nd_level index size}} = 2^7$



HW 10: Q 10-13

single-level Page Table

$EAT = h(a+m) + (1-h)(a+m+m)$

2-level Page Table

$EAT = h(a+m) + (1-h)(a+m+m+m)$

(PID, VPN)

(30, 1)

PageNo

0x1200

offset 12

4 KB = 2

⇒ offset : 12 bits

Frame no. : 3

3x200

3200

PID

1

(10, 7)

0x4500

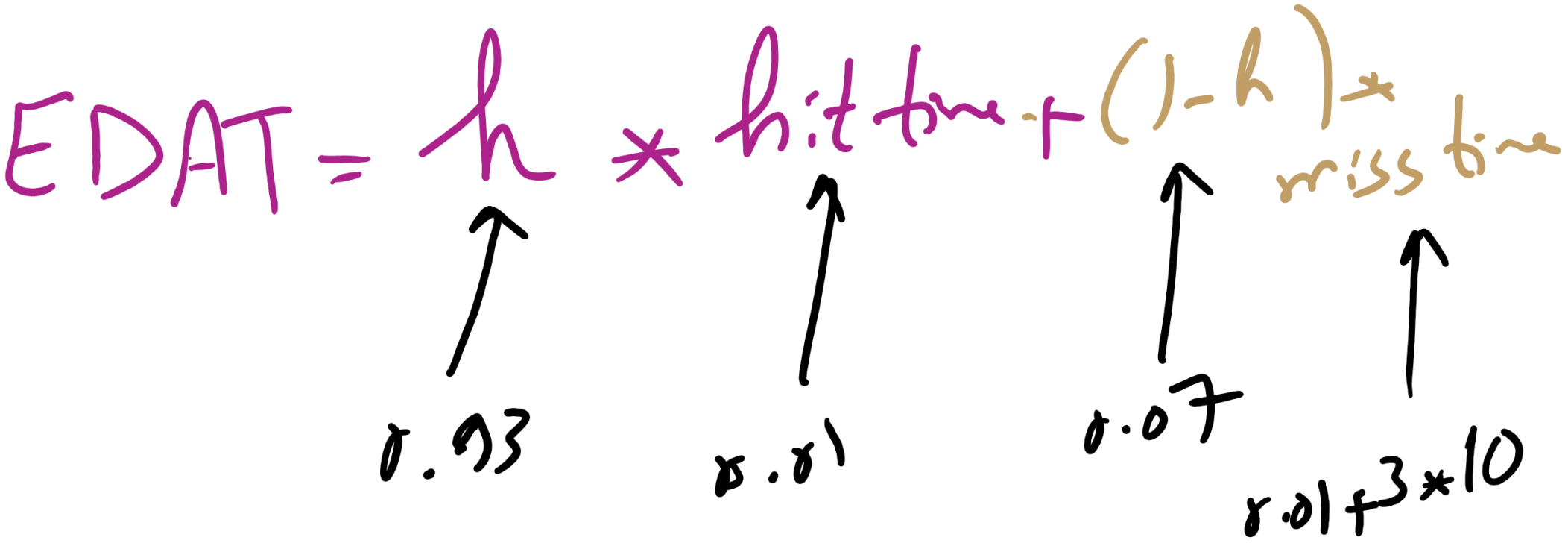
offset

Frame no.

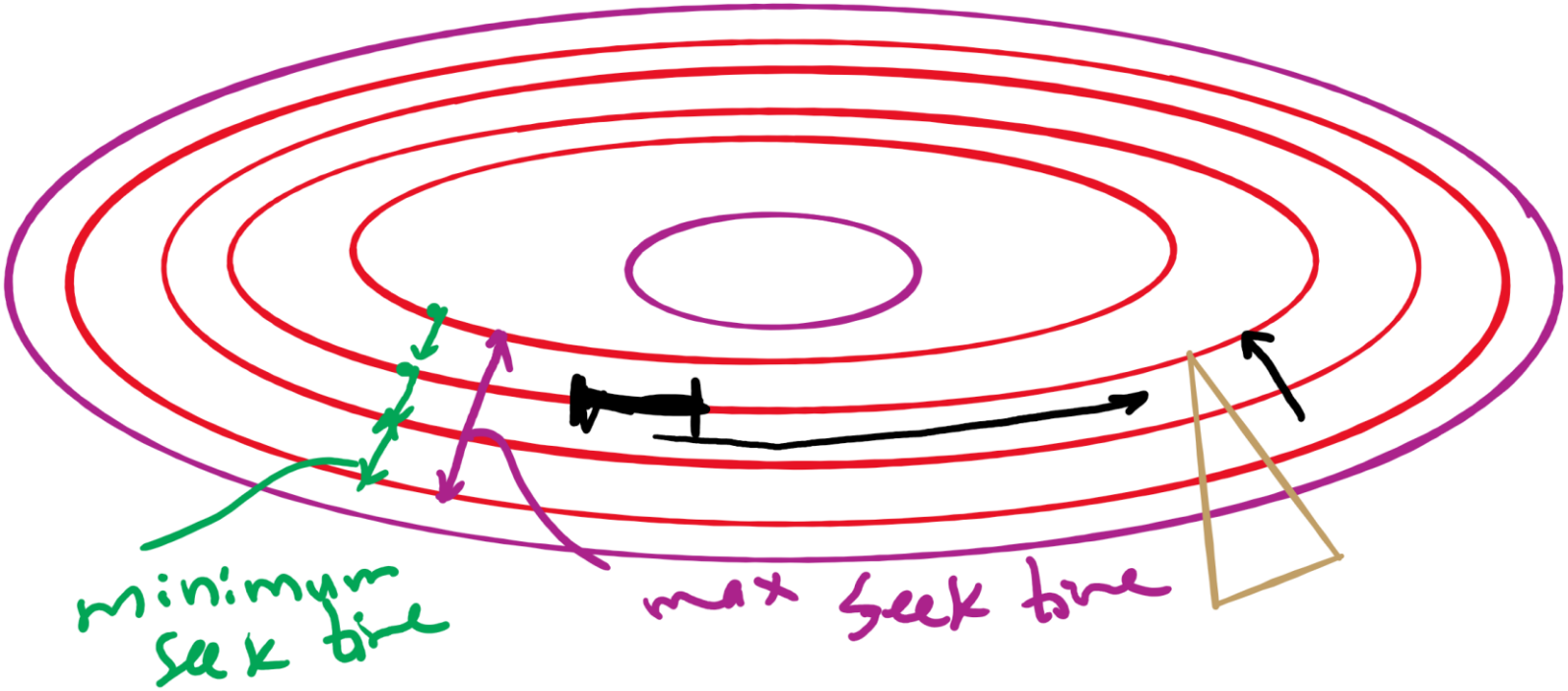
Frame no. : 4

0x7500

Effective Disk Access Time



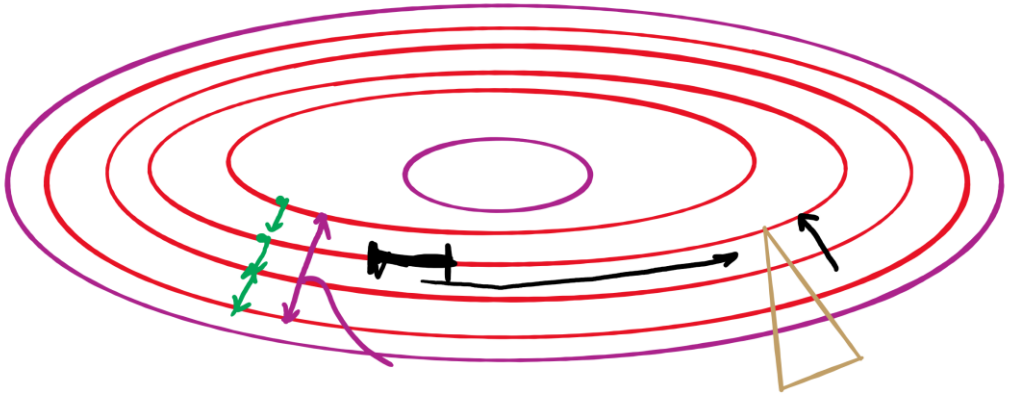
Minimum and Maximum Seek Time



max seek time = min seek time * (# tracks - 1)

cylinders

Average Rotational Delay

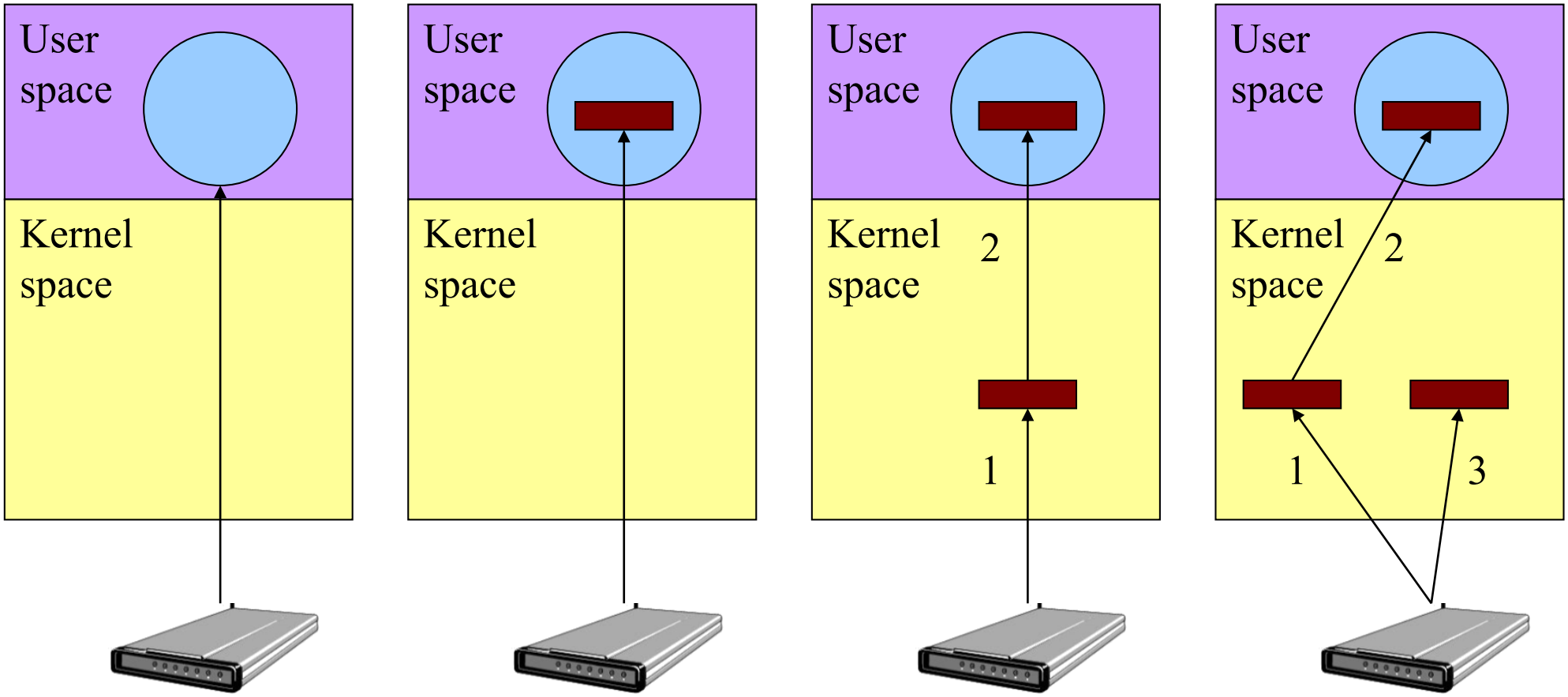


$$\text{Average rotational delay} = \frac{\text{one full rotation time}}{2}$$

$$= \frac{1/\text{rotation speed}}{2}$$

$$= \frac{(1/4800) * 60 * 1000}{2} \text{ ms}$$

Buffering device input



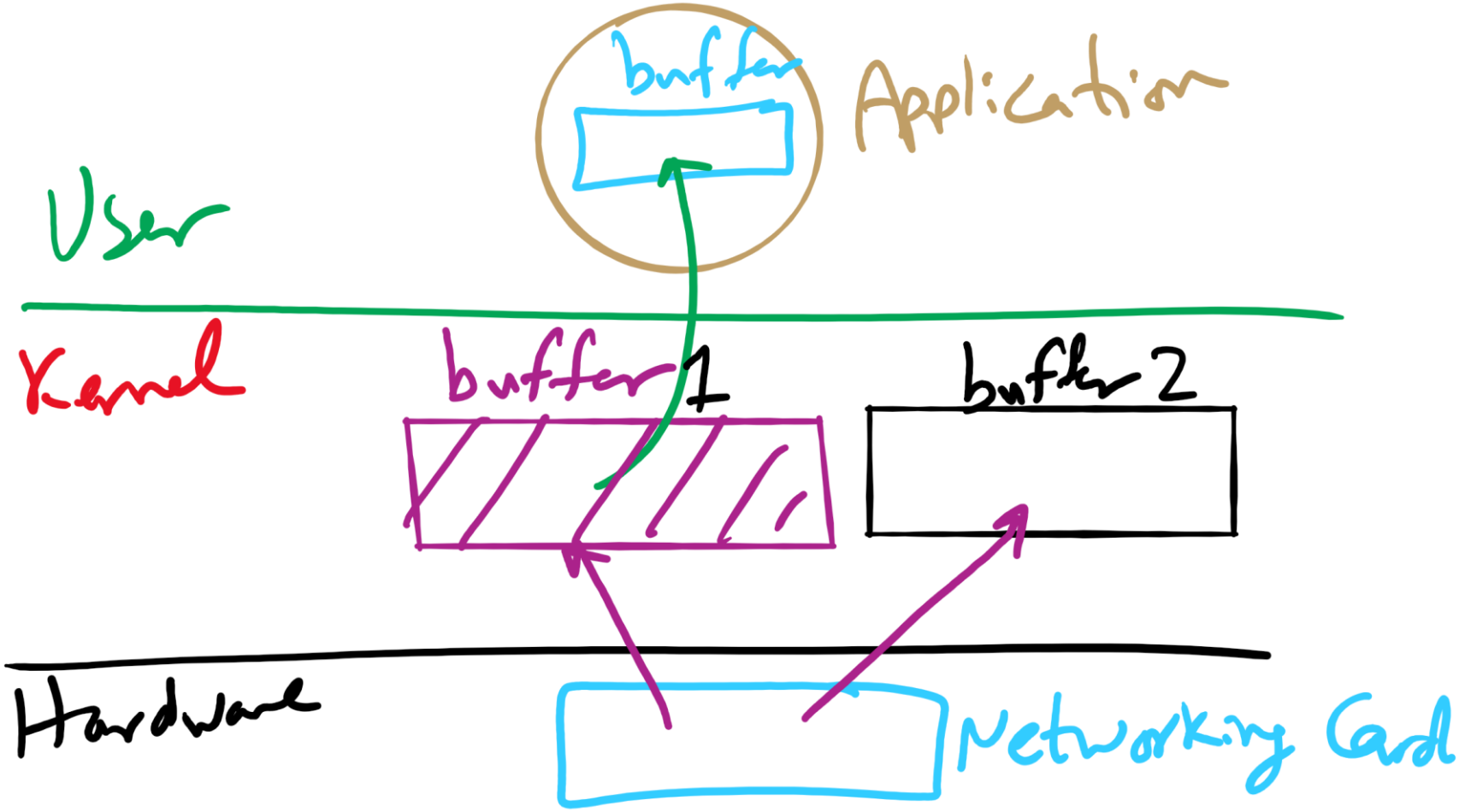
Unbuffered input

Buffering in user space

Buffer in kernel
Copy to user space

Double buffer in kernel

I/O Buffering

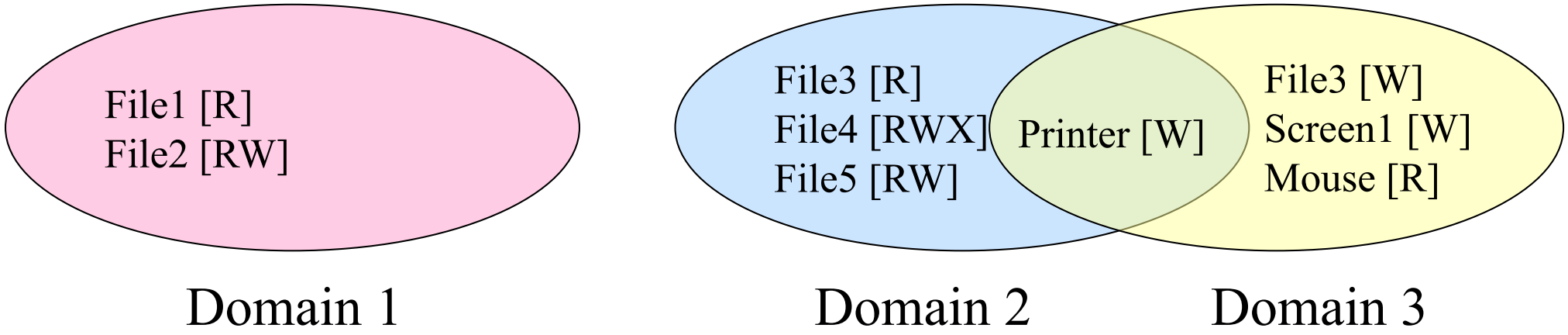


Problem of the Day: Protection

- Protection is about controlling access of programs, processes, or users to the system resources (e.g., memory pages, files, devices, CPUs)
  - How to decide who can access what?
  - Specifications must be
    - Correct
    - Efficient
    - Easy to use (or nobody will use them!)

Protection domains

- A process operates within a protection domain
  - resources accessible by the process
  - each domain lists objects with permitted operations
- Domains can share objects & permissions
  - Objects can have different permissions in different domains
  - There need be no overlap between object permissions in different domains
- How can this arrangement be specified more formally?



Protection matrix

	File1	File2	File3	File4	File5	Printer1	Camera
Domain1	Read	Read Write					
Domain2			Read	Read Write Execute	Read Write	Write	
Domain3			Write			Write	Read

- Each domain has a row in the matrix
- Each object (resource) has a column in the matrix
- Entry for <object, column> has the permissions
- Who's allowed to modify the protection matrix?
  - What changes can they make?
- How is this implemented efficiently?

Domains as objects in the protection matrix

Domain	File1	File2	File3	File4	File5	Printer1	Camera	Dom1	Dom2	Dom3
1	Read	Read Write						Modify		
2			Read	Read Write Execute	Read Write	Write		Modify		
3			Write			Write	Read		Enter	

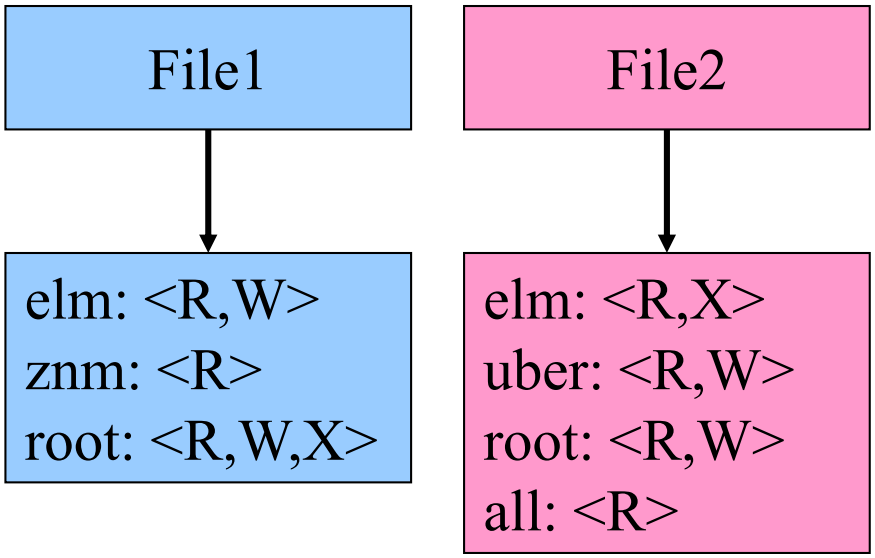
- Specify permitted operations on domains in the matrix
  - Domains may (or may not) be able to modify themselves
  - Domains can modify other domains
  - Some domain transfers (switching) permitted, others not
- Doing this allows flexibility in specifying domain permissions
  - Retains ability to restrict modification of domain policies

Representing the protection matrix

- Need to find an efficient representation of the protection matrix (also called the *access matrix*)
- Most entries in the matrix are empty!
- Compress the matrix by:
  - Associating permissions with each object: *access control list*
  - Associating permissions with each domain: *capabilities*
- How is this done, and what are the tradeoffs?

Access control lists (ACLs)

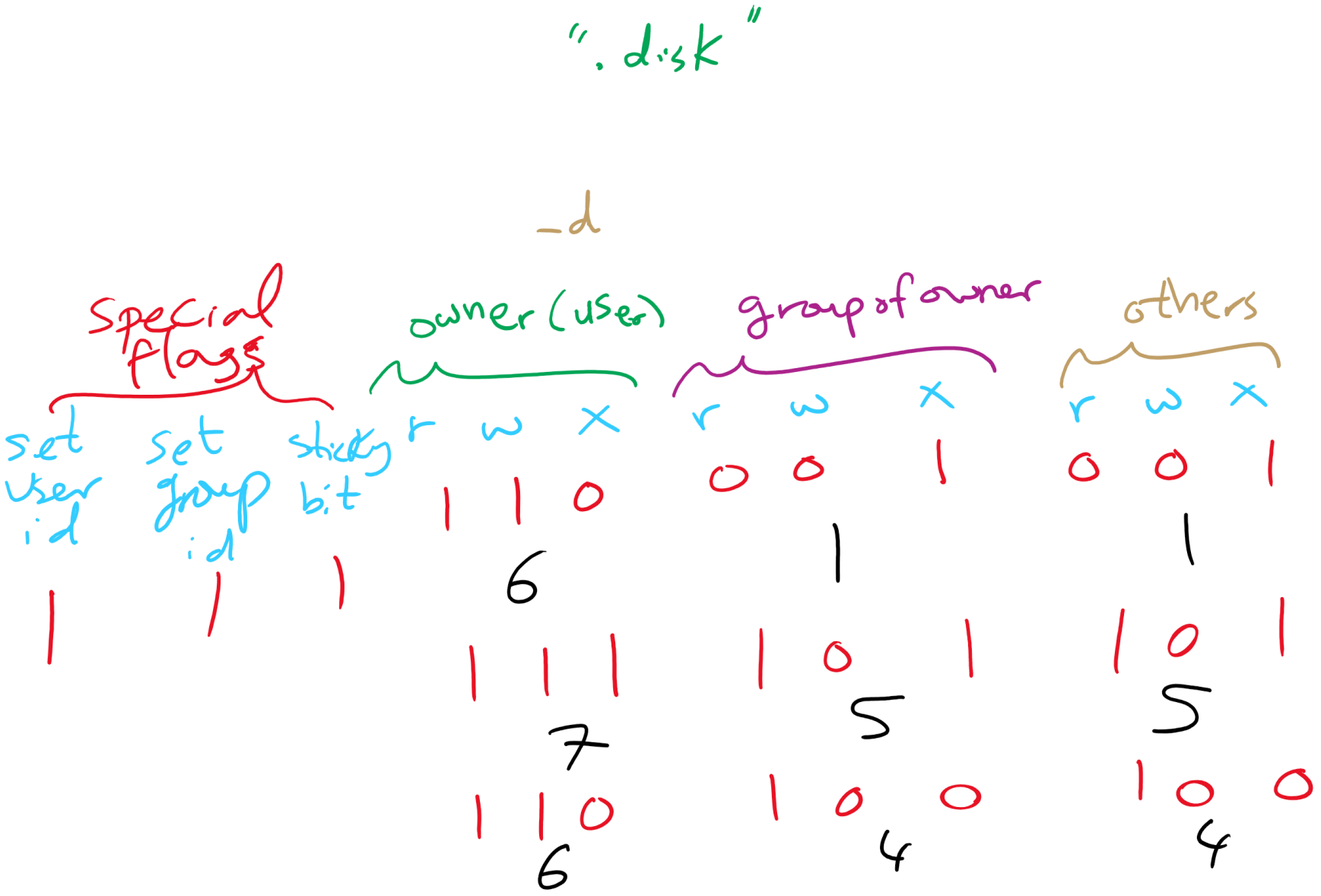
- Each object has a list attached to it
- List has
  - Protection domain (User name, Group of users, Other)
  - Access rights (Read, Write, Execute, Others)
- No entry for domain => no rights for that domain
- Operating system checks permissions when access is needed



Access control lists in the real world

- Unix file system
  - Access list for each file has exactly three domains on it
    - User (owner)
    - Group
    - Others
  - Rights include read, write, execute: interpreted differently for directories and files
- AFS
  - Access lists only apply to directories: files inherit rights from the directory they're in
  - Access list may have many entries on it with possible rights:
    - read, write, lock (for files in the directory)
    - lookup, insert, delete (for the directories themselves),
    - administer (ability to add or remove rights from the ACL)

ACL in UNIX


". disk"
_d
Special flags
set user id
set group id
sticky bit
owner (user)
group of owner
others
r w x
1 1 0
6
0 0 1
1
0 0 1
1
1 1 1
7
1 0 1
5
1 0 1
5
1 1 0
6
1 0 0
4
1 0 0
4